

Mi primer flujo en n8n

Jesús · 25 Sep 2026

Mi primer flujo en n8n: clasificador de mensajes con IA

Guía explicada paso a paso, como para niño chiquito. Proyecto:

C:\Users\jesus\n8n_clasificador.

1. ¿Qué es n8n? (la idea en una frase)

n8n es una mesa de dominós para computadoras. Pones fichas en fila, empujas la primera y cada ficha empuja a la siguiente. En n8n cada ficha se llama **nodo** y hace UNA cosa: recibir un mensaje, preguntarle a una IA, decidir algo, contestar...

Tú no programas todo desde cero: **acomodas cajitas y las unes con flechas**. Por eso se llama herramienta **low-code** (poco código). Make y Zapier son lo mismo pero de otras empresas; n8n tiene una ventaja grande: **lo puedes instalar en tu propia compu gratis** (con Docker), y ahí nadie te cobra por ejecución.

¿Para qué lo usan las empresas? Para no hacer a mano cosas repetidas: contestar mensajes, pasar datos de un formulario a Excel, avisar por WhatsApp cuando pasa algo, etc.

2. Las palabras que tienes que saber

- **Workflow (flujo):** la fila completa de dominós. El nuestro se llama "Clasificador de mensajes de clientes (IA)".
- **Nodo:** una ficha. Cada una tiene una tarea.
- **Trigger (disparador):** la PRIMERA ficha, la que arranca todo. Puede ser "cada lunes a las 9", "llegó un correo" o, en nuestro caso, "llegó un mensaje por internet".
- **Webhook:** una **dirección web** (como un buzón) donde otros programas dejan mensajes. El nuestro es `http://localhost:5678/webhook/mensaje-cliente`. Cuando alguien manda algo ahí, el flujo arranca.
- **JSON:** el formato en que viajan los datos. Es como una ficha de datos: `{"nombre": "Ana", "mensaje": "¿cuánto cuesta...?"}`.
- **Item:** cada "paquete" de datos que pasa de un nodo al siguiente.
- **Expresión `{{ }}`:** una fórmula, como en Excel. `{{ $json.nombre }}` significa "pon aquí el nombre que venía en los datos".

- **Ejecución (execution):** cada vez que el flujo corre. n8n guarda un historial y puedes ver qué entró y qué salió de cada nodo (esto sirve muchísimo para encontrar errores).
 - **Activo vs. prueba:** mientras lo armas usas la dirección de prueba (`/webhook-test/...`) y le das "Execute workflow"; cuando ya funciona lo **activas** y usa la dirección normal (`/webhook/...`) todo el tiempo.
 - **Variables de entorno / credenciales:** los secretos (como la llave de Gemini) NO se escriben dentro del flujo. Se guardan aparte y el flujo solo dice "usa la llave" con `{{ $env.GEMINI_API_KEY }}`. Así puedes compartir el flujo sin regalar tu llave.
-

3. ¿Qué hace nuestro flujo?

El negocio ficticio **Climas del Pacífico** (el mismo de DocuCita) recibe muchos mensajes de clientes. Alguien tendría que leerlos todos, ver de qué tratan y contestar. Nuestro flujo lo hace solo:

```
[1. Llega el mensaje]
|
[2. Gemini lo clasifica]
|-- falla --> [2b. Gemini de respaldo]
|               |-- falla --> [2c. Sin IA: pasar a humano] --> [6. Contestar]
|<-- funciona ----+
|
[3. Leer lo que dijo la IA]
|
[4. ¿Es urgente?]
|-- sí --> [5a. Pasar a un humano] ----+
|                                     +--> [6. Contestar]
|-- no --> [5b. Responder automático] -+
```

En palabras: **llega un mensaje → la IA dice de qué trata, si es urgente y redacta una respuesta → si es urgente se avisa a una persona; si no, se contesta solo.**

4. Nodo por nodo (qué hace y POR QUÉ)

Nodo 1 — "Llega el mensaje" (Webhook)

- **Qué hace:** abre un buzón en `/webhook/mensaje-cliente` que acepta mensajes tipo POST con `nombre y mensaje`.
- **Por qué un webhook:** porque así CUALQUIER cosa puede mandarle mensajes: un formulario web, WhatsApp, un chat de la página. El flujo no sabe ni le importa de dónde vienen.
- **Detalle importante:** está configurado en modo "Respond with *Respond to Webhook* node", o sea: "no contestes todavía; espera a que el último nodo tenga la respuesta".

Nodo 2 — "Gemini lo clasifica" (HTTP Request)

- **Qué hace:** le manda el mensaje a Gemini (la IA de Google) con instrucciones: "eres el asistente de Climas del Pacífico; dime la **categoría** (cotización, mantenimiento, garantía, queja u otro), si es **urgente** y escribe una **respuesta** corta y amable".
- **Truco clave: respuesta en JSON con esquema.** Le pedimos a Gemini que conteste SOLO con `{categoria, urgente, respuesta}` y le damos la lista de categorías permitidas. Así la IA no puede contestar con un rollo: siempre devuelve los mismos 3 campos, y los siguientes nodos pueden usarlos.
- **Reglas anti-inventos:** el prompt dice "NO inventes precios ni fechas; si hace falta, di que un asesor confirma". Es la misma idea que DocuCita: mejor admitir que no sabes que inventar.
- **Por qué HTTP Request y no un nodo "de IA" ya hecho:** porque con HTTP Request le hablas a CUALQUIER API. Si mañana cambias a OpenAI o Claude, cambias la dirección y listo. Además aprendes cómo funcionan las APIs por dentro.
- **Reintentos:** si Gemini falla (pasa seguido en el plan gratis: "modelo saturado"), el nodo reintentará una vez más (2 intentos, 20 s máximo cada uno). Si aun así falla, entra el plan B (nodo 2b).

Nodos 2b y 2c — el plan B y el plan C (agregados después de ver fallar a Gemini)

Probando el flujo, Gemini contestó **"Service unavailable" (503)**: el modelo gratis estaba saturado y el flujo se caía con "Error in workflow". Un cliente real se habría quedado sin respuesta. Por eso:

- **Nodo 2 tiene dos salidas.** En *Settings* → *On Error* se puso **"Continue (using error output)"**: si se acaban sus reintentos, en vez de tronar manda el mensaje por una segunda salida llamada *Error*.
- **2b. Gemini de respaldo:** es una copia del nodo 2 pero con **otro modelo** (`gemini-3.5-flash-lite`, variable `GEMINI_FALLBACK_MODEL`). Como un plan B: si un modelo está saturado, muchas veces el otro no.
- **2c. Sin IA: pasar a humano:** si el respaldo TAMBIÉN falla, un nodo Set arma una respuesta fija ("recibimos tu mensaje, un asesor te contacta en unos minutos") y marca el mensaje para una persona. **El cliente siempre recibe respuesta.**
- **Tiempos más cortos:** cada intento espera máximo 20 s y cada modelo tiene 2 intentos. Antes eran 60 s × 3 y una prueba tardó más de 2 minutos en fallar; en un chat eso es eterno.
- **Truco importante:** la salida de error NO trae el mensaje original, trae la información del error. Por eso los nodos 2, 2b y 2c leen el mensaje directo del nodo 1 con `$( '1. Llega el mensaje' ).first().json.body`.
- A esto se le llama **degradación elegante** (*graceful degradation*): si algo falla, el sistema sigue funcionando aunque sea de forma más simple.

Nodo 3 — "Leer lo que dijo la IA" (Code)

- **Qué hace:** Gemini devuelve su JSON como texto metido en `candidates[0].content.parts[0].text`. Este nodo lo saca, lo convierte en campos de verdad (`JSON.parse`) y lo junta con el nombre y el mensaje original.
- **Por qué:** para que los siguientes nodos puedan preguntar `{{ $json.urgente }}` directamente. Es el único pedacito con código (unas 15 líneas de JavaScript).

- **Además** agrega el campo `modelo` (principal o respaldo) usando `$( '2b. Gemini de respaldo' ).isExecuted`, para saber en cada respuesta qué modelo contestó.

Nodo 4 — "¿Es urgente?" (IF)

- **Qué hace:** una pregunta de sí o no: ¿`urgente` es verdadero? Tiene dos salidas: **true** (arriba) y **false** (abajo).
- **Por qué:** los mensajes peligrosos (chispas, olor a quemado) o clientes muy molestos no deben recibir una respuesta automática: los tiene que atender una persona ya.

Nodo 5a — "Pasar a un humano" (Set / Edit Fields)

- **Qué hace:** pone `accion = AVISAR A UN HUMANO YA` y cambia la respuesta por un mensaje fijo y seguro: "apaga el equipo desde el interruptor, un técnico te llama en menos de 1 hora".
- **Por qué fija y no de la IA:** en lo peligroso no se improvisa; el texto lo decide el negocio.
- **En la vida real** aquí se agregaría un nodo que mande un WhatsApp o correo al técnico de guardia.

Nodo 5b — "Responder automático" (Set / Edit Fields)

- **Qué hace:** pone `accion = respuesta automática` y deja la respuesta que redactó la IA.

Nodo 6 — "Contestar" (Respond to Webhook)

- **Qué hace:** le regresa al que mandó el mensaje todo el resultado en JSON: nombre, mensaje, categoría, urgente, acción y respuesta.
- **Por qué:** así quien llamó al webhook (una página, un bot de WhatsApp) recibe la respuesta lista para mostrarla.

5. Cómo lo instalé (paso a paso)

- 1 **Docker Desktop** abierto (el mismo que usa DocuCita).
- 2 En PowerShell, dentro de `C:\Users\jesus\n8n_clasificador`:
`.\iniciar_n8n.ps1` Esto descarga n8n la primera vez y lo arranca en **http://localhost:5678**. La llave de Gemini la toma de `docucita\.env` y se la pasa a n8n como variable de entorno (`GEMINI_API_KEY`). El modelo se elige con `GEMINI_MODEL`.
- 1 Abrir `http://localhost:5678` y **crear la cuenta de dueño** (solo vive en tu compu).
- 2 **Importar el flujo:** menú de tres puntos → *Import from File* → `clasificador_mensajes.json`.
(También se puede por consola: `docker exec n8n n8n import:workflow --input=...`)
- 3 **Activarlo:** el switch *Active/Inactive* arriba a la derecha.

6. Cómo probarlo

- Con el flujo activo: `.\probar.ps1` manda 3 mensajes de ejemplo (Ana pide precio, Carlos dice que su clima echa chispas, Lupita quiere agendar limpieza) y enseña qué contestó.
- Para verlo "por dentro": en n8n, pestaña **Executions** → abre cualquier ejecución y da clic en cada nodo para ver qué entró y qué salió. **Esto es lo que más impresiona en una entrevista:** se ve el camino que tomó cada mensaje.

Resultados reales (25-sep-2026, n8n 2.40.7 + gemini-flash-lite-latest)

Ana: "Hola, cuánto cuesta instalar un minisplit de 1 tonelada en mi recámara?"

- categoría: **cotizacion** · urgente: **no** · acción: respuesta automática
- respuesta: "¡Hola, Ana! Con gusto te apoyamos con la cotización. En un momento un asesor se pondrá en contacto contigo para darte todos los detalles sobre la instalación."
- Fíjate: NO inventó un precio, tal como decía la regla.

Carlos: "El clima que me instalaron la semana pasada está echando chispas y huele a quemado!!"

- categoría: **garantia** · urgente: **SÍ** · acción: **AVISAR A UN HUMANO YA**
- respuesta: "Hola Carlos, gracias por avisarnos. Por seguridad, apaga el equipo desde el interruptor. Un técnico te llama en menos de 1 hora."
- Fíjate: se fue por la rama de arriba del IF y la respuesta es la fija de seguridad, no la de la IA.

Lupita: "Quiero agendar una limpieza para mis 2 minisplits antes de que empiece el calor"

- categoría: **mantenimiento** · urgente: **no** · acción: respuesta automática
- respuesta: "¡Hola Lupita! Con gusto podemos agendar el mantenimiento de tus dos minisplits. Un asesor se comunicará contigo pronto para confirmar los horarios disponibles."

Los 3 casos cayeron en la categoría correcta y por el camino correcto.

Pruebas del plan B y plan C (25-sep-2026, tarde)

Para probar que el respaldo sirve de verdad, se arrancó n8n **a propósito con modelos que no existen** (`.\iniciar_n8n.ps1 -Modelo no-existe -Recrear`):

- **Principal roto, respaldo bien:** Rosa preguntó cuánto cuesta revisar un minisplit que no enfría → categoría **cotizacion**, respuesta de la IA, `modelo: respaldo`. El flujo no se cayó.
- **Los dos rotos:** respuesta en **13 segundos**: "Hola Rosa, recibimos tu mensaje. Un asesor te contacta en unos minutos." → `accion: PASAR A UN HUMANO` (la IA no respondió).
- **Todo normal otra vez:** Ana, Carlos y Lupita salieron igual que en la mañana.
- Extra: "se me está goteando agua del minisplit sobre el contacto de la luz" → **garantía, urgente**, pasa a humano.

Problemas que me encontré (y cómo se arreglaron) — esto también se cuenta en entrevista

- **Gemini 503 "Service unavailable":** el flujo se caía. Se agregaron el nodo de respaldo (2b), la red de seguridad sin IA (2c) y tiempos más cortos. Ver arriba.
 - **La lista de Executions salía vacía:** solo no había cargado; al recargar aparecen todas. Cada ejecución (verde = bien, roja = error) se puede abrir y ver qué pasó en cada nodo.
 - **"NOT NULL constraint failed: workflow_entity.id" al importar:** n8n 2.x exige que el archivo del flujo traiga un `id`. Se agregó `"id": "ClasificadorIA01"`.
 - **Activar por consola:** en n8n 2.x ya no es `update:workflow --active=true` sino `n8n publish:workflow --id=...` y reiniciar n8n.
 - **Acentos raros en la consola:** PowerShell 5 lee los scripts sin BOM como si no fueran UTF-8; se guardaron con BOM.
 - **Poca RAM (0.4 GB libres):** se apagaron los contenedores de DocuCita antes de arrancar n8n.
-

7. Cómo explicarlo en la entrevista

En español (30 segundos)

"Hice un flujo en n8n que atiende mensajes de clientes de un negocio de climas. Llega el mensaje por un webhook, Gemini lo clasifica con una respuesta en JSON con esquema fijo —categoría, si es urgente y un borrador de respuesta—, y un IF decide: si es urgente, como un equipo echando chispas, lo pasa a una persona con un mensaje de seguridad fijo; si no, contesta automático. Si Gemini falla, que me pasó con errores 503, el mensaje se va a un modelo de respaldo, y si ese también falla se contesta con un mensaje fijo y pasa a una persona: el cliente nunca se queda sin respuesta. La llave de la API va en variables de entorno y en Executions se ve el camino de cada mensaje."

In English (practice this one out loud)

"I built an n8n workflow that triages customer messages for an air-conditioning business. A webhook receives the message, Gemini classifies it using structured JSON output — category, whether it's urgent, and a draft reply — and an IF node routes it: urgent cases, like a unit sparking, go to a human with a fixed safety message; everything else gets an automatic reply. When Gemini returned 503 errors, I added a fallback: the node's error output sends the message to a second model, and if that one fails too, the customer gets a fixed reply and a human takes over — so nobody is left without an answer. I tested it by breaking the models on purpose. The API key lives in environment variables, and I can see every run in the Executions tab."

Palabras nuevas: *fallback model, error output, graceful degradation, timeout.*

Palabras en inglés útiles: *workflow, node, trigger, webhook, structured output, routing, branch, retry, human in the loop, self-hosted.*

8. Preguntas que te pueden hacer

- **¿Por qué pediste JSON con esquema?** Porque si la IA contesta texto libre, cada vez sale distinto y el IF no sabría qué leer. Con esquema siempre llegan los mismos campos.
 - **¿Qué pasa si la IA se equivoca y dice "no urgente" en algo urgente?** Es el riesgo principal. Mejoras: una regla extra sin IA (si el texto dice "chispas", "humo" o "quemado", urgente sí o sí), y revisar las ejecuciones para medir cuántas veces acierta —como hice con la evaluación de DocuCita—.
 - **¿Qué pasa si Gemini no responde?** Me pasó de verdad (503). El nodo reintenta; si sigue fallando, su salida de error manda el mensaje a otro modelo; y si ese también falla, se contesta con un mensaje fijo y se pasa a una persona. Lo probé apagando los modelos a propósito. En producción además agregaría un *Error Workflow* que me avise.
 - **¿n8n vs Make vs Zapier?** Los tres unen apps con cajitas. Zapier es el más fácil pero caro por tarea; Make es visual y barato, bueno para escenarios con ramas; n8n es open source y *self-hosted*, así que puedes correrlo en tu servidor, meter código cuando haga falta y no pagas por ejecución.
 - **¿Cómo lo conectarías a WhatsApp?** Cambiando el trigger por el de WhatsApp Business (o que el bot de WhatsApp llame a este webhook) y agregando al final un nodo que mande la respuesta por WhatsApp.
-

9. Ideas para mejorarlo (si te piden "¿y qué más le harías?")

- 1 Guardar cada mensaje en **Google Sheets** (un registro para el dueño del negocio).
- 2 En la rama urgente, mandar **WhatsApp o Telegram** al técnico de guardia.
- 3 Regla de palabras peligrosas antes de la IA (doble seguridad).
- 4 Conectarlo a DocuCita para que la respuesta use los precios reales del negocio con citas.
- 5 Un **Form Trigger** para tener una página con formulario y hacer la demo sin consola.